

Development Roadmap for the Ecosystem Simulation Game

Phase 1: Align Current Simulation with Vision (2D Foundation)

Assessing the Base Architecture: The existing Godot **2D simulation** provides a solid agent-based foundation, but we need to bridge it to the grand vision outlined in the concept docs. Key systems present in code and how they align or fall short:

- **Agent & Society System:** The code models individual agents with needs and behaviors (hunger, inventory, roles, etc.) – implementing an **agent-based model** as envisioned. For example, each agent has its own hunger and makes decisions in a deterministic tick loop. This aligns with the concept's call for agent-based simulation where each NPC "lives out their life" independently. However, the current decision logic is hard-coded (if-else rules) and not yet the **"novel AI brain"** the vision imagines. We have a place to insert improvements, but need to refactor the AI decision-making to be more modular so that a future LLM-powered cognition can plug in.
- **Economy & Crafting:** A rudimentary economy exists: agents earn and spend money, there's a **Market** class handling buy/sell orders with a matching engine, and a crafting system with workshops and recipes. The presence of currency, trade, and craftable tools (axes, pickaxes) echoes the vision of an emerging **player-driven economy**. The concept PDF describes features like a simulated economy with trade and currency (inspired by *Eco*), which our current code begins to address with its market and contracts. That said, it's still basic – only a few item types and recipes. To reach the vision, we'll need to **expand the crafting tech tree** (more resources, more manufactured goods) and possibly introduce concepts like skill or research. For now, the core is there, but it might need restructuring into more **data-driven** definitions (so new items/recipes can be added easily via config, which is partly done via JSON loading).
- **World & Environment:** The current world is a 2D grid (96×96 by default) with **resource nodes** (berries, trees, ore) that agents can harvest. There's a basic **regeneration** mechanism: each day, berry bushes and trees regrow some resources, and pollution is tracked per tile and decays over time. This is a simplistic stand-in for a true ecosystem. The vision calls for a **"fully simulated ecosystem"** with flora, fauna, climate, etc., which far exceeds the current implementation. We will need to modularize and extend the world model – likely moving from fixed resource nodes to dynamic entities (plants that grow/spread, animal populations that move and reproduce). The grid-based approach is a good starting point for determinism and ease of calculation, and we should keep it through early phases (it will also simplify multiplayer sync). In short, the environment system needs to evolve from a few regenerating nodes to a **high-fidelity ecosystem** model (incrementally – start with simple plant/animal logic in 2D before scaling up).
- **Law, Faction & Governance:** Impressively, the code already features **ownership and law enforcement systems**. Agents can claim land (tiles) and there are rules like requiring a permit to harvest on owned land. There's an `Enforcement` class that handles illegal actions, issuing fines and even market bans for violators. Additionally, agents can form factions when their grievance is high, creating a shared territory with its own laws and treasury. Factions have governance: the code

supports proposals and voting on law changes (e.g. adjusting permit requirements, taxes, or environmental laws). This aligns extremely well with the vision's focus on **collaborative governance and emergent social structure** (akin to Eco's player-run governments). The presence of this system is a strength we should build on. In the near term, we might keep it simple (maybe a couple of factions form as a demo), but eventually this can be a core feature (players/NPCs negotiating laws). One thing to ensure is that this system is **modular** – laws are data-driven and can be extended (the proposal system already allows changing various parameters via `changes` dictionary). We should continue to isolate the **faction/governance logic** so it can expand (e.g., more law types, integration with AI "opinions").

Short-Term Adjustments: Before adding new features, some **refactoring** is wise to better modularize the current code:

- **Separate Concerns:** Right now, the `Sim` class orchestrates everything in a single `_tick()` (handling market, contracts, agent actions, etc.). As we add complexity (especially with AI brains and a 3D world), we should ensure each subsystem (ecology, economy, AI, etc.) can be updated independently. For example, we might separate the "environment update" (resource regrowth, pollution spread) from the "agent update". The current daily update already does some of this (resources regrow daily, contracts and factions update daily). We can build on that by perhaps introducing a clearer **system update order** (e.g., Environment → Agent Decisions → Actions/Movement → End-of-day summary). This will help if we need to throttle or tweak specific systems (important for performance scaling).
- **Configurable Tuning:** The `tuning.json` and `items.json` provide a lot of knobs to adjust behavior. We should continue using tuning parameters to avoid hard-coding values. As we implement new features (e.g., animal reproduction rates, weather frequency), expose them in the tuning config. This makes it easier to achieve the vision's balance through iteration and to enable players/modders to customize the world.
- **Visualization Decoupling:** Currently, visualization is minimal (probably a debug view in Godot). As we move to 3D, we must **keep simulation logic separate from rendering**. For now (in 2D), ensure that the simulation can run without any GUI – this makes it easier to run headless servers or automated tests. Godot's scene graph for 3D will be mostly a view on the simulation state (e.g., a tree entity in the simulation corresponds to a Tree node in the scene). We should start designing how to map simulation data to visuals cleanly (perhaps via a mapping of resource types to scene templates).

By the end of Phase 1, we aim to have the current 2D sim cleaned up and **better aligned to the vision**: all essential systems (agents, economy, factions, world) identified and structured, ready to be expanded. It will still be a simple 2D "sandbox" at this stage, but with a clear map of what needs to be added or reworked. Notably, we'll be set up to start integrating the "AI brain" and richer ecosystem next, without the legacy code fighting us. The idea is to have a robust base where we can confidently say: the prototype already captures determinism, basic ecology, economy, and governance – now we can **iterate toward depth and scale** in subsequent phases.

Phase 2: Introduce LLM-Powered Agent Cognition (AI “Brain” Integration)

Goal: Evolve NPC behavior from the current scripted approach to a more **dynamic, life-like intelligence** by integrating lightweight Large Language Models (LLMs) or similar AI techniques. This will be done incrementally – preserving the existing decision structure as a fallback or for less important agents, while giving some agents a new “brain” module.

- **Modular Brain Architecture:** We will design the agent AI as a **pluggable module**. Instead of each agent using `decide_action()` with fixed logic, we introduce an interface where an agent queries its “brain” for decisions. For now, the default brain can just call the old `decide_action` (to keep things working), but we can swap in an LLM-based brain for certain agents or certain decision types. The concept suggests a hybrid approach: *“the NPC brain likely shouldn’t rely solely on an LLM”*. We’ll follow that by breaking the brain into layers or components, some symbolic and deterministic, others probabilistic or learned. Key layers to implement:
- **Memory:** Each agent will maintain a memory store – facts about the world and past experiences. Currently, agents have some state (inventory, last action, grievance, etc.), but we’ll extend this to a more narrative or semantic memory that an LLM could use. For example, an agent could remember “I was fined by Faction A yesterday” or “We are low on food this winter.” Technically, this could be a list of key events or a summary string that’s updated periodically. This aligns with giving the LLM context to work with so it’s not stateless. We might start by logging important events in a text form for a single test agent.
- **Perception (Environment Context):** When asking an AI model to choose an action, we need to feed it what the agent perceives right now. We will construct prompts or structured inputs that include things like time of day, location, nearby entities, and the agent’s current needs/goals. For example: *“It is evening. You are at the market. You are hungry (hunger=20%). You see 2 other villagers here. Berries are scarce in the world.”* This can be generated from the simulation state each tick (or each time we query the AI). We’ll need to be mindful of token limits and only include salient details – perhaps use a short description for nearby environment and a few memory items.
- **Personality & Goals:** Each agent already has some personality variables (risk_tolerance, eco_concern, grievance). We will leverage these to condition the AI’s responses. For instance, an agent with high eco_concern might have an LLM prompt prefaced by “You are an environmentally conscious person who hates pollution.” This helps the AI produce decisions consistent with character. Similarly, risk_tolerance could influence whether the AI suggests risky actions like stealing. The concept of “novel AI brain” highlights combining simple models with higher-level cognition ¹ – here, personality traits and deterministic rules can guide the LLM, effectively narrowing its choices to sensible ones.
- **Decision-Making & Planning:** Rather than letting the LLM decide completely freely, a practical approach is **hybrid planning**: use classical AI or our existing logic to generate a set of feasible actions, and let the LLM choose or elaborate on them. For example, if an NPC is hungry, our rule-based system knows the options are “eat berries, eat meal, or obtain food (by gathering or trading).” We can present the LLM with something like: *“You are hungry. Options: 1) Go gather berries from the forest, 2) Go to the market to buy food, 3) Eat food you have.”* and have the LLM pick one based on character context (“You remember being fined for illegal harvest, so you avoid the forest... go to market”). This way, the LLM isn’t hallucinating totally new actions outside the game’s scope; it’s working within a constrained action set, adding *“believable human-like reasoning”* to the choice.

Initially, we might test this with one agent in a controlled scenario to see if the outputs make sense, and keep using the deterministic `decide_action` for others.

- **Dialogue & Interaction:** One exciting aspect of LLM-driven agents is natural language interaction. While not a core gameplay loop yet, we can start laying groundwork: e.g., an LLM agent could narrate its decision ("I should go to the market, it's safer.") or even chat with the player. We should ensure the architecture allows **communication**: agents could have an `intention` or `thought` that can be surfaced. In multiplayer, this means sending text to clients for AI dialogue. This can remain off by default in the prototype but is a key long-term goal (imagine players negotiating with AI mayors or traders in-game).
- **Incremental Integration Plan:** We won't flip all NPCs to LLM overnight – too costly and potentially chaotic. Instead:
 - **Experiment in Isolation:** Prototype the LLM brain with one agent in a sandbox. For example, have a single AI-driven NPC in a test scene who has a simple goal (gather food) and run a cheap LLM (perhaps a local model or an API with a very concise prompt) to decide its actions for the day. This step is about proving the concept. As the concept doc suggests, *"take one agent and experiment with a cheap LLM for its decisions or dialogue"*. We'll observe if the LLM's choices can be made consistent and reasonable. We may discover we need to fine-tune the prompting or incorporate more rules.
 - **Embed into Simulation:** Once we have a working prototype of the AI brain, integrate it into the main simulation. We might assign the LLM-brain to a subset of agents (say, village leaders or a few "hero" NPCs) and keep others on the old system to save computation. The agent code will check what brain module it has – one that uses `OpenAI` (for example) vs the default.
 - **Optimize Calls:** Use **asynchronous processing** for LLM queries. The simulation tick is deterministic and runs fast; an LLM call might be relatively slow (e.g., 100ms or more) and could vary in time. We cannot block the whole sim waiting. A solution is to have the AI brain run on a separate thread or even separate server process. For instance, when it's time for an LLM-agent to act, we send an AI request (with current state, etc.) to an AI service. While we wait for the reply, the agent could either do a default safe action (like idle or a short deterministic routine) or we simply skip its turn for a tick. When the response comes back, we apply it (e.g., the agent decides to move to the market). This way the simulation isn't stuck, and we can handle multiple AI calls in parallel if needed. We must also **handle failures or timeouts** – default to the rule-based behavior if the AI is unavailable, so the game remains robust.
 - **Cost Management:** Use **lightweight** models or rate-limit the calls. Perhaps we use a small local model (for example, fine-tune LLaMA-2 7B or use GPT-3.5 with short prompts) for each agent's decision. Also consider that not every tick needs a fresh AI decision – an agent could make a multi-step plan and stick to it for several ticks. For example, if the LLM decides "I will go to the forest to gather wood," that plan might occupy the next 5–10 ticks of movement and gathering. We can then refrain from calling the LLM every single tick, greatly reducing overhead. This approach of **LLM for high-level planning** and using simpler execution logic for low-level actions will make integration much more efficient.
 - **Multiplayer Architecture for AI:** We will run the AI computations on the **server side**. In a multiplayer game, we can't have each client running full AI for NPCs (that would desync and be wasteful). Instead, the authoritative server will determine NPC actions (with its AI module) and broadcast the results to clients. Essentially, the server is the "brain center" for NPCs. Clients just receive the outcomes (e.g., NPC X moved here or said this line of dialogue). This keeps the heavy lifting centralized and ensures consistency. We might design a dedicated thread pool or even a

separate microservice for AI, so the game server remains responsive. In early prototypes, the AI can be part of the server process, but modularizing it (so it could be scaled or even run on a different machine) will future-proof the design if we scale up the number of LLM-driven agents.

By the end of Phase 2, we expect to have **at least a rudimentary LLM-driven agent in the game**, demonstrating more human-like decision-making. The “brain” architecture will be in place, though not all NPCs will use it yet. We’ll likely still rely on our rule-based AI for the majority of agents (both for performance and because it’s a good deterministic backup). But we will have proven the concept of an AI citizen that can plan, remember, and perhaps even converse in a way that static scripts can’t achieve. This sets the stage to gradually expand this system – tuning the AI’s intelligence and integrating it with in-game systems (for example, an AI agent reacting to ecological data or proposing a law change based on its LLM reasoning). It’s a big step toward the vision of NPCs that *“behave in human-like ways”*, powered by a **novel cognitive architecture**.

Phase 3: Upgrade to a High-Fidelity Ecosystem Simulation

With agents and AI improving, we now turn to the **ecosystem** – evolving from the minimal resource model to a richer, more realistic environment. The vision describes a living world with plants, animals, climate, and complex feedback loops (like over-harvesting causing extinctions). Phase 3 will introduce these elements step by step, ensuring we keep things computationally manageable.

- **Introduce Flora Diversity & Life Cycle:** Currently, “plants” are just static resource nodes (Berry bushes, Trees with a stock count). We’ll transform these into entities with simple life cycles:
- **Implement growth and reproduction:** Instead of a fixed daily stock regen, have each plant grow over time and potentially spread. For example, each tree could have an age and size; over a certain period, it produces a seed that can create a new sapling on a nearby tile (if space and conditions allow). This can be probabilistic (e.g., each tree has a X% chance per day to spread a seed to an adjacent tile). Berries could be seasonal: they might bloom and fruit once a year (or simply have a growth rate that is faster in certain seasons). We might maintain the simplicity of a **grid-based approach** for now: each tile could hold at most one plant entity (or a small number), to make it easy to manage and avoid overpopulation.
- **Multiple plant species:** Down the line, we can introduce different plant types (with different nutrition values, growth rates, biome preferences, etc.), but for the prototype stage, even a single tree and single crop type is enough to validate the mechanics. Ensure these are driven by data (so adding new species later is just adding to a JSON or resource file).
- **Environmental factors:** Add simple abiotic factors that affect plants. For instance, **soil fertility** or moisture could be a property of each tile (perhaps initialized randomly or based on a noise map for variability). This could influence growth probability (e.g., a tree spreads faster on fertile tiles). We could also include **elevation** or terrain type if we generate a 3D world – e.g., some tiles are water (no plants except aquatic ones), some are rock (hard for plants to grow). In Phase 3 we won’t simulate full climate systems yet, but we can stub in variables like temperature and rainfall and see their effects in a limited way. *Example:* define that the world has a “winter” every 100 ticks during which berry growth is 0. This introduces a seasonal cycle as mentioned in the concept (predictable rhythm for environment).
- **Introduce Fauna (Animal Simulation):** Adding animals will bring the **food web** to life. We start small:

- **Herbivores:** Introduce a herbivore species (e.g., deer or rabbits) that lives in the world. We can treat animals similarly to agents or as a separate class of simulation entity. Since animals won't use the complex economic systems, we might create a simpler `Animal` class with properties like hunger, a position on the grid, and a routine to move and eat. Herbivores will seek out plants (e.g., berry bushes) to eat. When they eat, they reduce the plant's stock or maybe kill the plant (if we want to simulate overgrazing). They reproduce at some rate if well-fed. We can simulate a basic **predator-prey dynamic** by just having herbivores and plants: herbivores increase when plants are plenty, overeat them, then herbivores starve, plants regrow – an oscillating cycle.
- **Carnivores (Predators):** If time permits or for a mid-term enhancement, add a predator species that hunts the herbivores. This will add another layer to the cycle. Predators would reduce herbivore numbers, giving plants relief, etc., creating classic Lotka-Volterra dynamics. We should carefully keep numbers small at first to avoid huge performance costs. Possibly simulate animals in an **aggregated way**: instead of hundreds of individual rabbits, we might simulate a herd or population number per area. For example, each tile or region could track an approximate animal population that grows or shrinks based on rules. This is less granular but much cheaper than simulating each animal AI. We'll choose the approach based on performance measurements – maybe a hybrid (simulate a few representative animals as entities, track the rest as numbers).
- **Animal AI:** If treating animals as entities, give them simple decision logic (not LLM, just reactive). They move randomly or toward food, eat, and flee predators. This can be done with a few rules – effectively similar to how agents move to resource nodes but without the economy. It should be deterministic and use the same RNG for consistency across clients.
- **Deepen Pollution & Environment Interaction:** Currently, pollution comes from mining ore (each ore gathered adds pollution to that tile), and pollution affects hunger (increased hunger drain globally based on average pollution). This is a good start to link economy to environment. We will expand pollution to have more **ecological consequences**:
 - High pollution could slow plant growth or kill plants (e.g., if a tile's pollution > 0.8, maybe nothing grows there) – simulating toxicity.
 - Pollution could have a radius effect (spread to adjacent tiles slowly, representing water/air flow). Possibly implement a simple diffusion each tick from polluted tiles to neighbors.
- **Climate change aspect:** Track a global pollution index (or specifically CO₂ if we separate types of pollution later). If it exceeds a threshold, we could adjust climate variables – e.g., increase global temperature. A simple effect of higher temperature might be faster plant growth but also faster water evaporation (if we simulate water). Or it could be purely negative, like certain crops yield less. We might incorporate events like “drought” or “flooding” tied to climate – this might be later, but keeping the door open for such **random events** is part of the plan (random weather events are mentioned as a realism factor).
- We should also include **resource depletion**: If players/agents over-harvest (cut all trees in an area), those resources might not come back easily. The game Eco has the concept of possible extinction or permanent depletion. We can simulate something akin by having plants require a seed source nearby to regrow. If an entire species in an area is wiped out with no seeds, it won't magically respawn. This means players/agents would have to import seeds or wait a long time. It introduces consequences for overuse, fulfilling the vision of an ecosystem that *can be irreversibly damaged by mismanagement*. In the short term, we can keep regeneration generous, but gradually make it more realistic as other systems (like replanting or conservation by agents) come online.
- **Ecosystem Feedback into Gameplay:** Ensure these new ecosystem elements tie back to agents and factions:

- If animals eat crops, that directly competes with agents' food supply – creating potential conflict where agents might decide to **hunt animals** or protect farms. We could allow agents a new action (e.g., a hunter role that can gather meat from animals). This adds another economic resource (meat, leather, etc.) and intertwines with ecosystem.
- Factions or laws might regulate hunting or pollution. For example, if wildlife drops too low, perhaps a faction proposal emerges to ban hunting for a time. The framework for laws is already in place to do things like that (we can extend the Laws/Proposals to include environmental regulations – e.g., “set a hunting quota” could be a proposal type in the future). Initially, simpler: we at least feed data into faction grievances or proposal logic. Indeed, the code already hints at ecological awareness: in `_create_proposals`, they check `avg_pollution` and `food_scarce` to trigger certain law proposals (like stricter harvest permits if pollution is high, or lowering taxes if food is scarce to encourage trade). We will build on this by adding more such triggers (e.g., if an animal population crashes, perhaps an agent with high `eco_concern` gains grievance, prompting action).
- Use the **metrics system** (or expand it) to monitor ecosystem health. The current `Metrics.create_snapshot()` likely records things like total resources, population, etc., each day. We should include new metrics: number of each animal, number of trees alive, average temperature, etc. These can be displayed to players (a hallmark of Eco is exposing data to players for informed decisions). In our case, this could simply be a debug graph at first. But having the data is crucial for balancing and for AI to potentially read it. For instance, an AI agent might be coded or prompted to read “wildlife population = 5 (very low)” and decide to propose a hunting ban.
- **Performance and Complexity Management:** A high-fidelity ecosystem can become CPU-intensive. To keep the simulation **scalable** as we add these features, we'll implement a few strategies (some already noted above):
- **Spatial Partitioning:** The grid itself is a partition, but if we have many entities, we may only update those near active agents or players frequently. For example, if part of the map is uninhabited, we can update its plants/animals at a slower tick rate (or even pause simulation there until someone comes near). This “limiting scope” approach is hinted at in the concept doc (dynamic update scaling for large worlds). Godot's processing can be managed by only instantiating objects that are needed. We might keep far-away entities as simple data and only create active objects when in range.
- **ECS and Batching:** We will consider refactoring some of the simulation to an **Entity-Component-System** style for efficiency. In practice, this could mean storing all animals in a list and updating them in a tight loop (which we essentially do now in GDScript, but we might optimize critical loops in C# or C++ modules if needed). We should also use fixed-point or deterministic math for any physics if introduced, to maintain determinism across platforms (the concept mentions this for multiplayer sync), though in pure GDScript logic with our RNG, we should be fine.
- **Probabilistic rules with fixed seed:** We've already embraced randomness in a controlled way (using a custom RNG with a seed ensures reproducibility). We'll continue that: ecological events (births, deaths, weather) can be random but driven by our RNG, so all clients see the same outcome. This yields natural variability without losing sync. We just must be careful to use the RNG consistently and not rely on any nondeterministic calls.

By the end of Phase 3, the simulation should feel much more **alive** and true to the vision of a working ecosystem. We should be able to observe scenarios like: unchecked mining causing pollution that slows plant growth, which then causes animal populations to starve, which in turn might trigger famine for agents – *illustrating an interdependent web*. We'll likely need to fine-tune parameters to keep the world in a somewhat stable range (avoid total collapse unless intentionally provoked) – achieving a balance between growth and consumption. This phase sets the stage for the **3D world and multiplayer** by ensuring we have

all major gameplay systems (ecology, economy, social, AI) in place in 2D. The next step will be to bring this world to players in a networked 3D environment.

Phase 4: Transition to Godot 3D & Multiplayer Architecture

This phase brings our simulation from a single-player 2D testbed into a **multiplayer 3D game**. It's a significant leap, but by doing it after solidifying core mechanics, we ensure the game's depth carries over. Key tasks in this phase:

- **Godot 3D World & Rendering:** We will move the project to use Godot's 3D capabilities. The simulation will still run on the same logic (grid-based world, ticks, etc.), but we'll introduce a 3D representation for it.
- **World Representation:** Start with a simple **flat terrain** divided into a grid corresponding to our simulation tiles. For example, a 96×96 heightmap or plane, where each tile is perhaps 1x1 meter (or 1x1 unit). We might generate a procedurally textured terrain (grass, soil, water) and overlay a grid for reference. We don't necessarily need a spherical planet like Eco for now – a flat square map is far simpler technically. Biomes can be faked by varying ground texture or plant types in different regions (e.g., more trees in the “north” part = forest biome).
- **Entity Models:** Create or import basic 3D models for game entities:
 - Trees, berry bushes, etc. – We can use low-poly tree models for forest, perhaps a bush model for berries. On each simulation tick, if a resource node exists at a tile and is “alive,” ensure a corresponding MeshInstance or MultiMesh is displayed. If the node is depleted or cut down, remove or swap the model (e.g., a tree stump model to show it was cut).
 - Agents – Give players and NPCs a simple human avatar (even a capsule or primitive with a sprite billboard would do initially). The model should have an indicator of faction or player (maybe colored clothing) if possible, or at least a name tag. Agents will move on the grid; we can smoothly interpolate their movement in 3D for visual polish. Godot's animation or even a NavMesh could be used for nicer movement, but since they move tile by tile, it might suffice to animate a translation over a half-second between tile centers.
 - Animals – If we have animals from Phase 3, give them models (e.g., a deer model). They can use similar movement interpolation.
 - Buildings/Workshops – Represent workshops or other structures with 3D models (a small house or a workbench model at the tile). The **market** at (market_pos_x, market_pos_y) could be depicted as a marketplace stall or town center model.
- **Camera & Controls:** Implement a 3D camera that allows players to view the world. Possibly an RTS-style camera (pan/zoom) to oversee the simulation, and/or a first-person/third-person camera if players control a character directly. Given Eco's style, a first-person or third-person view for players could be immersive. Initially, a free-fly camera or orbit camera might be simpler for debugging. Eventually, if players have avatars, we'll want a character controller (Godot provides a KinematicBody or CharacterBody we can use) so players can walk around, interact with objects (like pressing a key to gather a resource if near).
- **UI in 3D:** We can use Godot's 2D GUI system in a canvas layer on top of 3D to show things like inventory, health, chat, etc. Also, some in-world UI might be useful (labels over characters' heads, or signs on claimed land). Start with minimal HUD: maybe just the player's hunger bar and money, and a simple way to open inventory or metrics screens.

- **Multiplayer Networking Architecture:** Decide and implement the networking model to allow multiple players and possibly remote server hosting. Based on research and the project's needs, the **Authoritative Server** model is preferred for our game: one instance (server) runs the simulation and clients connect to it.
- **Authoritative Server Implementation:** We can use Godot's high-level multiplayer API. For example, run a Godot instance in dedicated server mode (no rendering) that loads the simulation scene. This server will be the only one running `_physics_process` (tick updates). Clients will connect and receive updates. We'll need to mark certain nodes/state as replicated or manually sync via RPCs. A straightforward approach: designate the `SimState` or relevant parts as networking master on the server, and broadcast critical state to clients at intervals.
- **State Synchronization & Determinism:** Because our sim is deterministic, we *could* use lockstep (each client runs the simulation too and we sync inputs). But as noted earlier, lockstep for a large, continuous sim with many agents is risky and hard to scale beyond small groups. Instead, we use the server's state as authoritative. However, we can still exploit determinism: the server can send only minimal info needed and trust clients to interpolate or even predict some outcomes. For instance, we might send random seeds for upcoming events, or have clients simulate minor cosmetic behaviors (like animal idle animations) without needing server confirmation, knowing they won't diverge in important ways. The concept notes that determinism is mainly a tool for fairness and replay/debugging, so our approach will be: **trust the server, but keep the simulation deterministic under the hood** to catch issues. Clients that join mid-game will get a full snapshot of state to initialize. We might implement a checksum system (the code already has a `Sim.checksum()` function for state) to verify clients haven't diverged, which is useful for debugging desyncs.
- **Networking Data & Frequency:** Decide how often and what data to sync:
 - **Agent positions and states:** Probably the most frequently updated. We can send these perhaps every simulation tick or every few ticks. Position, health/hunger, current action could be sent for each agent. We might compress this (only send what changed, or use area-based updates). Since $96 \times 96 = \sim 9216$ tiles and maybe a few dozen agents, sending all positions isn't huge. With more agents it could be, so later we'll add **interest management**: only send data of agents/objects near a player's location or within camera view. Godot doesn't have built-in interest management, but we can script it (e.g., partition world into regions). For the prototype, broadcasting everything to all might be fine, but plan to optimize.
 - **Resource and world changes:** Changes to resource nodes (depletion, regrowth, pollution changes) can be sent less often. Perhaps we send a world state snapshot every in-game hour or day. Or whenever a resource node a player might see changes, we trigger an update. If players have a world data UI, we might send the summarized metrics periodically instead of every little detail.
 - **Player inputs:** Clients will send actions to the server (e.g., "I want to move here" or "I harvest this"). The server will validate (e.g., is there a tree to harvest, does this player have a permit?) and then update the sim state, then inform all clients of the results (like tree removed, inventory changed). This keeps the authoritative rule. We should be mindful of security – authoritative model already prevents cheating mostly, but we should still validate everything server-side.
- **Multiplayer in Godot specifics:** We'll use **RPCs or the new MultiplayerSynchronizer** in Godot 4. Possibly, we can designate the `SimState` as a synchronized object and use RPC calls for player actions. For example, when a player wants to perform an action, we call an RPC to the server's

function, which then runs the same logic as an NPC agent performing that action (we could even unify player and NPC through the same `Agent` class, which it already is – `is_player` flag). The server, after processing ticks, can use `rset` or RPCs to update clients. Using Godot's built-in mechanisms where possible will simplify development (e.g., setting an agent node's position property to sync). But due to our custom simulation loop, we might end up doing more manual syncing (which is okay for clarity).

- **Testing Sync and Lag:** We will thoroughly test with multiple clients to ensure the game stays in sync. We should simulate latency and packet loss in tests (Godot's network API can help fake latency) to ensure our approach (especially if not lockstep) doesn't lead to weird issues. We expect small discrepancies in non-critical things might occur, but as the concept says, we can "let minor differences slide by always trusting the server's state". If a client diverges, the server will correct it on the next update. The determinism and our use of integer math for most logic (and seeded RNG) mean there should be very little divergence anyway.
- **Player Interaction & UI Improvements:** In multiplayer, players are now part of the world. We need to implement how they **play**:
- **Controlling Agents:** Easiest is to allow each player to effectively "possess" an agent (perhaps the one marked `is_player=true` that we already spawn). That agent then follows player inputs instead of AI decisions. For example, when a player presses movement keys or clicks, we set their agent's target accordingly (overriding `decide_action`). This way, the player is just an agent with a human brain. The rest of the simulation treats them the same (they get hungry, can craft, etc.). We already spawn one player agent in `init_new()` – we can expand that to one per connected client.
- **In-Game Actions:** We'll provide UI or controls for things like gathering, crafting, trading, law voting:
 - Gathering: If a player is near a resource (within 1 tile), we can show a prompt "Press [E] to harvest [Tree]" for example. On pressing, send action to server, execute similarly to NPC gather (and subject to same law checks).
 - Crafting: If at a workshop, open a simple crafting menu (list of recipes, highlight those you have materials for). Let player select a recipe to queue – this triggers the same logic as NPC queueing a craft. Possibly make it instantaneous for player or still require time – depending on design.
 - Trading: The market system is a bit complex (order books). For prototype, we might simplify by allowing direct trade or basic buy/sell to a market NPC. But if we keep it, we need a UI for market orders (show current prices, allow player to place an order for X item at Y price). This can be as simple as a text-based console or a rudimentary form. The simulation will handle the matching and either fulfill or queue the order.
 - Contracts: If contracts exist (quests to deliver items), we could show available contracts in a UI and let players accept them similar to NPCs. This might be lower priority for the first playable, but it's a unique feature we have in code that could engage players.
 - Laws/Voting: Potentially, allow players to vote on proposals if they are in a faction. This requires a UI to show active proposals. We might not have time in the prototype phase, but we ensure the system supports it. Possibly at least show the current laws in effect (so players know if harvesting is illegal somewhere).
- **Feedback to Players:** Because the game is complex, the UI should give feedback: e.g., if you try to harvest and it's illegal, a message "You don't have a permit to harvest here!" could appear (the enforcement system would trigger a fine behind the scenes, but the player needs to know what

happened). If pollution is killing crops, maybe a warning like “The soil is polluted.” These small UX touches will greatly help playability. We can leverage the data we have: the enforcement code can return reasons for denial (it currently just blocks silently by returning false on illegal gather – we might modify it to inform the agent/player).

- **Aesthetic and Performance Considerations:** With multiplayer and 3D, ensure the **client performance** is managed. Use Level of Detail and culling – e.g., don’t render every tree at full detail across the map. Godot can handle a few thousand objects, but if we simulate thousands of plants/animals, consider using multi-meshes for dense vegetation and only animating nearby ones. Also, as mentioned in the concept, if NPC AI is heavy, ensure clients aren’t trying to run it (we’ll keep that server-side). The clients mainly render and handle player input. We might also **throttle client updates**: if our tick is very fast (say 10 ticks per second), we might only send/receive at 5 Hz to reduce network load, and rely on interpolation client-side. These details can be fine-tuned in testing.

By the end of Phase 4, we aim to have a **networked 3D version of the game** running: multiple players can log in, see the world in 3D, move around, and interact with the environment and NPCs. The authoritative server ensures everyone sees a consistent world. We will likely run a dedicated server during development to test with several clients and refine synchronization. This phase is one of the most technically challenging, as predicted in the concept docs, because it merges all systems with networking and rendering. We’ll proceed methodically: first get a simple sync of positions working in a blank 3D world, then layer on features (resource sync, inventory sync, etc.). With this in place, we have essentially transformed the project into a **rudimentary MMO-style simulation game** – the core loop playable with friends, and all major components in play.

Phase 5: Deliver a Minimal Playable Prototype (Milestone)

At this stage, we consolidate the work into a **playable prototype** – the first version of the game that can be experienced end-to-end by a player. This prototype will be limited in scope but should illustrate the vision in miniature. We focus on a short-term milestone that demonstrates the following features in a 3D multiplayer environment:

- **Basic World & Ecology:** A small 3D world (for example, a **single valley or island** map) with a variety of resources present. We include:
 - Berry bushes and trees that regrow (or reproduce) over time, as our flora.
 - Possibly one animal species (e.g., wild deer) roaming that the player can observe (and maybe hunt, if we include that interaction).
- Pollution effects if applicable (for prototype, we might intentionally trigger pollution via some action like mining to show its effect).

The ecological rules should be active but tuned forgivingly so the world doesn’t collapse during a short play session. The goal is to let players see that **resources are finite but renewable** – e.g., if you cut all trees, you have to wait for regrowth or things change (and maybe see deer population drop if no plants left).
- **Core Gameplay Loop:** The prototype should answer “what does the player do?” in a simple loop:
- **Gather** – Players (and NPCs) can gather basic resources like berries for food and logs for materials. This involves exploring the environment a bit, finding resource nodes, and using a gather action.

We'll ensure the **controls and feedback** for gathering are clear (e.g., approach a tree, press a key to chop, see log added to inventory and tree model disappear or shrink).

- **Craft** – Include a couple of crafting recipes to transform resources, giving a sense of progression. For example, **Crafting planks from logs**, and **crafting a simple tool (like an axe) from wood and stone/metal**. The existing recipe system supports tools like Axe and Pickaxe. We might start with just planks and one tool, as those are already defined (axe requires planks + metal, which might be too advanced for a quick demo unless we also allow smelting ore to metal ingots). Alternatively, **cooking** could be a crafting example (berries to a cooked meal). The crafting process should be visible – e.g., use the workshop model, perhaps require some time (or we may make it instant for simplicity). This shows the **value-adding economy** aspect.
- **Trade** – Demonstrate the economy by allowing trading of goods. Since we have a market system, we can enable players to sell excess goods and buy needed ones. For the prototype, to keep it simple, we might spawn an NPC trader or use the market order system behind a basic UI. Example: a player can put up 5 planks for sale and an NPC (or another player) buys them, money changes hands. The **price system** (supply/demand) can be shown in a simple way (maybe as fixed prices or a basic dynamic price printed to screen). This introduces the idea of an **in-game currency and economic decisions**.
- **Survival/Needs** – Include the **hunger mechanic** to drive players to engage in the loop. Players and NPCs should need to eat periodically. Berries and cooked meals will restore hunger. This creates a win/lose condition of sorts – if you don't secure food, you could starve (perhaps we'll allow starving to reduce health or activity to show consequence). The hunger system is already in place (hunger drops each tick, eating restores it). We'll surface this to the player via a hunger meter UI and perhaps warnings if low. This gives players a tangible goal: get food to survive, and maybe make better food (cooked meal) which is more nutritious, illustrating progression.
- **NPC Integration:** Include a handful of **NPC agents** alongside the player. They will follow their AI routines (some possibly with the new LLM brain, others with default logic as needed). This makes the world feel alive and also demonstrates emergent behavior without player input:
 - NPC gatherers go out and collect resources (maybe competing with players for berries – prompting either cooperation or competition).
 - NPC crafters produce goods and possibly sell them. For example, an NPC crafter might collect logs and craft planks, then sell planks at the market. If the player needs planks to craft an axe, they could buy from that NPC via the market – demonstrating **specialization and trade** benefits.
 - NPCs should obey the same laws – if the player claims a piece of land (we might allow the player to claim a land tile if they can afford it, using the existing claim system), NPCs without permission won't harvest there, or if they do, the enforcement system fines them. This can show the **law and order** aspect. (For simplicity, we might script one NPC to claim a small area and enforce a permit law, just to let the player experience needing a permit or facing a fine. It could be a tutorial-like element: "NPC_Alice has claimed the orchard; if you pick fruit there, you might get in trouble.")
 - Optionally, showcase faction formation: if we can trigger it, having NPCs form a faction (maybe after some grievance event, like an environmental disaster or repeated fines) would be a great emergent story to observe. But if not in this milestone, it's okay – the groundwork is there for later.
- **Multiplayer Functionality:** Ensure that two or more **players can join** and play together in this prototype world. They should be able to see each other's movements, trade or share resources (drop

items or trade via market), and perhaps collaborate (one focuses on gathering while another crafts, for example). The server should handle concurrency (multiple actions at once). We'll test edge cases like both players trying to harvest the same resource – the server should allow only one to succeed (first-come, first-serve, the other gets nothing because the node depleted). These tests will validate our sync model. If any desync issues appear, we'll add fixes (e.g., locking mechanisms, or slight delays on actions to resolve conflicts).

- **Minimal UI/UX:** The prototype's user interface should tie everything together in an **accessible way**:
 - A **HUD** showing player stats (hunger, money) and possibly a mini-map or coordinate.
 - An **Inventory screen** to see what items you have (and use/eat items from it).
 - Basic **interaction prompts** (when looking at an object or NPC, show a key prompt like "F: Talk" or "E: Harvest"). Even if the "talk" just triggers a one-line dialog from an NPC, it adds immersion.
 - A **market interface** (even if text-based): list of buy/sell offers or a way to perform a transaction. We might simplify by having a fixed shop or NPC vendor for prototype instead of full order book UI – depending on what's faster to implement. The key is players can exchange goods/money somehow.
 - Feedback messages: e.g., "You are too hungry to work" if hunger too low, "You gained 5 Coins", "Illegal action! You were fined 10 Coins." from enforcement, etc. These can be small popup texts or chat log entries. They help the player understand the consequence of their actions in the complex simulation.

Prototype Success Criteria: A player (or small group) should be able to **play for around 15-30 minutes** and experience a microcosm of the game's promise: manage basic survival, see the economy functioning, witness the ecosystem responding (resources depleting/regrowing, maybe animals wandering), and interact with at least one AI-driven character who exhibits non-trivial behavior (could be as simple as an NPC that the player can follow and see it making decisions like a real person would). If the prototype includes an LLM-driven NPC, a great moment could be the player asking them a question (via chat) and the NPC responding in character – that would be a "wow factor" showcasing the AI. Even if not, the interplay of systems (ecology/economy/governance) should be evident.

We will gather feedback from this prototype: Was the game loop fun or frustrating? Did the ecosystem feel alive? Was multiplayer smooth? This will guide adjustments. Importantly, this milestone also doubles as a **tech demo** to potentially show stakeholders or team members that the vision is achievable: "Here is a tiny village in our world where players and AI coexist and everything from tree growth to trade to law enforcement is running procedurally." From here, we can plan content expansion and polish.

Phase 6: Ensure Modularity and Scalability for Long Term Growth

With a working prototype in hand, we turn to the future: scaling up and adding content without breaking the system. The final phase focuses on **architectural principles and refactoring** to keep the project sustainable as we head towards a full-scale game.

- **Embrace Modular Design Patterns:** As we add features, we should continuously refactor into modular components. Some guidelines:
- **Entity-Component-System (ECS) or Similar:** The simulation can benefit from ECS practices for performance and organization. We might not rewrite everything in a formal ECS, but we can simulate it. For instance, handle all "hunger" updates for agents in one place, all "movement" in

another, rather than having logic spread out. The concept specifically mentions ECS for scaling to thousands of entities. Godot 4 doesn't natively use ECS for gameplay, but we can structure our code to approximate it (e.g., have systems in GDScript that operate on arrays of agents or animals). If performance demands, consider moving critical loops to C# or C++ GDNative for speed.

- **Loose Coupling via Signals/Events:** Godot's signal system can help reduce tight coupling. For example, instead of the `Actions.execute_action` directly calling into many subsystems, we could emit events like "AgentPerformedHarvest" with context, and have the Enforcement system listen to decide if it's illegal, the World listen to update resource, etc. This way, adding a new system (say, an achievement system or an analytics logger) can be done by attaching to signals rather than modifying core logic. In general, events make the architecture more **extensible** – new features can plug in without rewriting the old. We should apply this where practical (but not over-engineer to the point of obscuring logic; balance is key).
- **Data-Driven Expansion:** We've touched on this, but to reiterate: many game elements should be defined in external data or configuration files. Items, recipes, species stats, faction policy defaults, etc., are ideally not hard-coded. The code already loads JSON configs for items/recipes. We should extend that approach to **ecosystem parameters** (e.g., define species in a JSON with fields for reproduction rate, lifespan, etc.), and **AI parameters** (the personality profiles for NPC archetypes could be data-driven). This not only makes tweaking easier (no recompiling to change a value) but also sets up for modding support. Modders could add a new animal or craft recipe just by adding JSON and a model, if our systems are generic enough. We'll keep an eye on designing systems to be content-agnostic (e.g., the crafting system already is generic, supporting any recipe that's defined; similarly, the law system is somewhat generic with proposals able to toggle any defined law parameter).
- **Scalable Networking:** As we prepare to possibly increase player counts or world size, consider how to segment the world. An **MMO architecture** might require splitting zones across servers or instancing. While that's beyond our prototype, making sure our code can handle partial worlds is useful. For example, ensure that if we only simulate a portion of the world on one server, it won't break assumptions. Using well-defined APIs for "world queries" (e.g., `World.get_node_at(x,y)`) means we could swap out the backend of `World` to query a distributed database or a region-specific store without changing agent code. This is speculative, but worth keeping in mind if the game's ambition is large scale. In the medium term, simply optimizing the current server (multithreading, efficient data structures) will likely suffice for dozens of players and thousands of entities.
- **Performance Tuning:** We will likely iterate on performance as we scale up content. Some focus areas:
- **Profiling and Load Testing:** Use Godot's profiler or custom timers to find slow spots. For example, if pathfinding is introduced later for more complex movement, that might slow things down. Or if our LLM calls increase, they might become a bottleneck. We should periodically test the sim with, say, 100 NPCs and 1000 animals to see how it holds up. Also test with high player count (simulate players by bots perhaps) to see network load. This will inform where to put engineering effort (e.g., optimize networking messages or reduce frequency of certain updates).
- **Adaptive Tick Rates:** We can consider running different subsystems at different tick frequencies. For instance, the core tick could remain, say, 10 Hz for agent actions, but the **ecosystem growth** might only update at 1 Hz (or even 0.1 Hz for slow processes like climate). Godot allows setting different processing intervals or using timers for less frequent updates. This way, we aren't wasting

CPU on checking plant growth every tick if it only changes daily. This ties into the spatial considerations: areas with no players could update environment on a slower cycle. The challenge is to do this while preserving determinism – but if it's just a function of time (e.g., “update this region every 10th tick”), that can remain deterministic as long as all clients know the rule.

- **Memory and Persistence:** As the world grows, so does state size. We should ensure our save/load system can handle it (the JSON serialization might become huge). We might move to a database or binary format eventually. But as a modular practice, we might isolate the persistence logic so it's easy to swap out (e.g., have an interface `SimulationStorage` with implementations for JSON, for SQL, etc.). In the short term, ensure our save function writes efficiently (perhaps compress the JSON or limit how often we auto-save in multiplayer). Persistence also means handling server crashes gracefully – maybe in later development we add periodic autosave and a way to reboot from last save.
- **Cross-Platform Determinism:** If we allow clients on different OS or hardware, floating-point or RNG could differ. We've mostly used our own RNG (which is good). We might further enforce determinism by using fixed-point math for critical calculations (for example, using integers for hunger instead of floats, or snapping floats aggressively as we already do when serializing). The current code's use of `snappedf` suggests we're aware of floating-point precision issues. We'll continue that practice anywhere nondeterminism could creep in. This makes the simulation robust in multiplayer and even for replay/debug.
- **Enhancing Content Modularly:** Once the architecture is stable, adding content (more items, more species, more laws, a tech tree, etc.) will be the focus. We should plan how to do this without needing engine changes:
- **Technology Progression:** The vision likely includes advancing technology (like Eco's meteor objective). We might introduce research or skill systems. These should ideally be independent modules (e.g., a `ResearchSystem` that ticks along, consuming resources to unlock new recipes). We can integrate it by having agents or players contribute to research projects. This system can listen to events (e.g., “player built a research bench”) and update some unlocks. Keeping it separate means we can cut it or modify it without affecting core simulation.
- **User Modification and Community:** Encouraging a community to form around the game can drive longevity. We plan for **mod-support** by designing the game data-first. For example, if a community member wants to add a new plant type, they could add to a JSON and drop a model in a folder, and the game would recognize it (assuming we code it to load all entries generically). For AI mods, since LLM prompts/behavior could be data-driven (one could edit the prompt templates or memory content), we might allow loading external scripts for AI or at least external prompt configs. We must be careful with security in multiplayer (server should enforce game logic, so mods might be cosmetic or server-side only). But having an open architecture means down the line players might script their own NPC behaviors or scenarios, which aligns with the **sandbox ethos** of the game.
- **Polish and Quality:** As we plan to scale, don't forget to **refine what's there**: fix bugs, improve AI logic edge cases, make UI more intuitive, etc. Some specific improvements once the base is working:
- Better pathfinding for agents (maybe they can navigate around obstacles in 3D, using Godot's NavMesh).

- More sophisticated AI personalities – e.g., some NPCs might be aggressive exploiters, others eco-friendly, leading to interesting faction dynamics (one faction over-harvests for profit, another retaliates with strict laws – an emergent story).
- Audio and visual feedback to make the world immersive: sounds for chopping trees, animal noises, a changing skybox for time of day, etc. These don't affect simulation but add to player experience. We should ensure adding them doesn't interfere with logic (run them on clients side only, etc.).
- **Testing and Debugging Tools:** As the simulation grows complex, invest in dev tools: maybe a debug menu to spawn resources or fast-forward time, a way to visualize the AI decision process (for example, log the reasoning from the LLM to console), and visualization of invisible data (like an overlay that color-codes pollution levels or ownership on the ground). These will help us (and eventually players, through mods or an inspector UI) understand the state of the world and troubleshoot issues. The deterministic nature allows interesting possibilities like replay – we could save the random seed and initial state, and later replay the entire game to pinpoint what happened (this was mentioned in concept as useful for debugging complex systems). While full replay might be long-term, even having the option to dump state snapshots and compare will be valuable.

By adhering to these modularity and scalability principles, we ensure that as we go from a dozen agents to hundreds, or from a small map to a large world, the game remains maintainable and performant. Each system (AI, ecology, economy, multiplayer) can be developed further by different team members without stepping on each other's toes, because we've defined clear boundaries and data interfaces. The architecture is **future-proofed** to incorporate new tech: for example, if a more efficient AI model comes out, we can swap the brain component; if we decide to use a different engine's network library for better performance, our networking layer is abstracted enough to replace, etc.

Ultimately, by the end of Phase 6, we will have transitioned from a scrappy prototype into a well-structured platform on which the **full vision can be realized**. The game can then be expanded with confidence: adding content (more of everything – species, items, societal features) and improving fidelity (better AI, better graphics) without needing to rewrite the core. This phased roadmap – going from tightening the current 2D sim, to layering AI, ecology, 3D, multiplayer, and then polishing the architecture – takes us from the current state to the ambitious vision of a **scalable AI-driven ecosystem simulation game**. Each phase builds on the last, and by following it, we ensure both **immediate progress** (a playable game early on) and **long-term sustainability**, so the project can grow in complexity and player count to meet its ultimate goals.

Sources Cited: The roadmap above is informed by the analysis of the current GDScript codebase and concepts from the design documents. Key connections include maintaining determinism for multiplayer fairness, using a hybrid AI approach for NPC brains, simulating detailed ecology with flora/fauna and feedback loops, and adopting an authoritative server model for networking. These references guide our development steps to ensure the implementation aligns with the envisioned features and scale of the game.

¹ Building a Scalable AI-Driven Ecosystem Simulation Game.pdf

file:///file-XR1fmatDGcvqB7ah6jMrxf